

ADVANCED MACHINE LEARNING · WORKSHEET 02

The ML Project Lifecycle (Part 2)

Data Isolation · Feature Engineering · Training Loops · Evaluation & Deployment
Lecture 2

Name:

Branch / Year:

Date:

TODAY'S ROADMAP - PHASES 5–11

Lecture 1 covered Phases 1–4 (gray). Today = Phases 5–11 (indigo).

1–4	Problem → Collection → EDA → Preprocessing (Lecture 1)
5	Data Splitting - The Golden Barrier
6	Feature Engineering - Crafting Perspective
7	Baseline Establishment - Setting the Floor
8	The Modeling Loop - Select, Tune & Train
9	Offline Evaluation - The Final Exam
10	Model Deployment - Into the Wild
11	Post-Deployment - The Living System

Parameters V/S Hyperparameters

Parameters are the internal gears adjusted automatically during training. Hyperparameters are external dials you set before training. The algorithm cannot adjust them on its own.

Type	Adjusted By	When	Example
Parameter	The model (automatically)	During training	Weights (w_1, w_2), Intercept (w_0)
Hyperparameter	The engineer (manually)	Before training	max_depth, n_estimators, learning_rate

Common Production Hyperparameters:

- **Decision Trees:** max_depth - maximum splits allowed. Too low = too simple; too high = memorizes data.
- **Random Forests:** n_estimators - how many trees to build in the ensemble.
- **Gradient Descent:** learning_rate - step size toward minimum error. Too large = overshoots; too small = never converges.

Hyperparameter Tuning is the iterative process of running multiple training cycles with different configurations, checking the score against the Validation Data, and finding the optimal combination.

▷ PRACTICE P1 · CLASSIFY: PARAMETER OR HYPERPARAMETER?

Label each item as [P] Parameter or [H] Hyperparameter:

- | | |
|---|--------------|
| (a) Weight w_1 in a regression equation | [P] |
| (b) <code>max_depth</code> of a Decision Tree | [H] |
| (c) Intercept w_0 in $y = w_0 + w_1x_1$ | [P] |
| (d) <code>learning_rate</code> for Gradient Descent | [H] |
| (e) <code>n_estimators</code> in a Random Forest | [H] |
| (f) Bias term learned during backpropagation | [P] |

PHASE 5 DATA SPLITTING (THE GOLDEN BARRIER)

▷ HOOK

If a student memorizes the answers to last year's exam, getting 100% does not prove they understand the subject. If your ML model scores 100% on data it has already seen, does it actually know how to predict the future?

Now that we understand why splitting matters (Parameters need training data, Hyperparameters need validation data, and the final grade needs untouched test data), let's formalize the three blocks.

The Data Splitting Framework:

Training Data ~70%	Validation Data ~15%	Test Data ~15%
<i>The data for model training</i>	<i>For validation (High Detail only)</i>	<i>The data for model evaluation</i>

△ WARNING SENIOR ENGINEER'S WARNING: DATA LEAKAGE

Data splitting must occur BEFORE any Feature Engineering. If you scale or impute using the full dataset first, information from the validation and test sets will leak into training. This gives artificially high scores but causes catastrophic failure on truly unseen production data.

★ KEY INSIGHT

If you peek (prematurely looking at data) at the test data during development, you have leaked the exam answers, and your final performance metrics are compromised forever.

▷ PRACTICE P2 · FILL IN THE BLANKS & TRUE/FALSE

- | | |
|--|------------------|
| (a) The <u>Test</u> set is locked away until the very end to give an unbiased final grade. | |
| (b) You should calculate the mean of the ENTIRE dataset before splitting to replace missing values. [True / False] | [False] |
| (c) Which set is used to tune Hyperparameters? <u>Validation</u> | |
| (d) Peeking at the test set during development invalidates your evaluation. [True / False] | [True] |

△ REFLECT 1

A data scientist computes the mean salary from ALL 100,000 rows, then splits. They use that to impute missing values in the training set. Is this Data Leakage? Why?

Hint: Did information from the Test set influence the Training set?

Expected answer: Yes, it is Data Leakage. Information about the test data distribution (the overall mean) influenced the training set, meaning the model implicitly learned from the test data.

◆ TAKEAWAY

Split FIRST, engineer SECOND. The Test Set is sacred - touch it only once, at the very end.

PHASE 6 FEATURE ENGINEERING (CRAFTING PERSPECTIVE)

▷ HOOK

Imagine predicting exam scores. One night before exam, one student studies 10 hours but sleeps 0 hours - they fail. Another sleeps 10 hours but studies 0 hours - they also fail. Success depends on the combination of study AND sleep, not either one alone. Can an algorithm figure this out if it only looks at each column separately?

Raw data is like raw marble. Feature Engineering is sculpting it so the underlying structure becomes obvious. It has two operational pillars:

1. Feature Creation

The process of generating entirely new inputs from existing ones to expose hidden relationships.

- **Leveraging Domain Knowledge:** Using industry expertise to turn raw values into meaningful concepts.
 - Example: Instead of processing raw temperature (41°C), use meteorological thresholds to group into: Cold ($\leq 15^\circ\text{C}$), Warm (16–30°C), Hot ($> 30^\circ\text{C}$).
 - This reduces infinite decimal noise into meaningful categories the model can exploit.
- **Combining Features:** Mathematically combining columns to capture joint effects.
 - Example: $\text{Water_Availability_Index} = \text{Average_Rainfall_Inches} \times \text{Soil_Moisture_Percentage}$
 - This single calculated value represents the combined effect directly to the model.

2. Feature Selection

If you throw 500 features at an algorithm, many of which are useless noise, the model will get confused and overfit. Feature selection keeps highly predictive inputs and discards the rest.

- **Filter Methods:** Screen features before training using statistical metrics (e.g., correlation). If a feature has correlation ≈ 0 with the target, filter it out.
- **Embedded Methods:** Feature selection happens automatically during training. Tree-based models (Random Forests) calculate internal Feature Importance Scores and tell you which variables mattered.

★ KEY INSIGHT

If your features are high quality, even a simple algorithm makes highly accurate predictions. If your features are poor, even the most complex neural network will struggle.

▷ PRACTICE P3 · FEATURE ENGINEERING APPLICATION

(a) [Interaction Term] Create an interaction feature for a house dataset with "Length" and "Width":
New feature: Area = Length × Width

(b) [Match the Concept] Match each scenario (1–3) to the technique (A–C):

- | | |
|--|-----------------------|
| 1. Dropping columns with 0.05 correlation to target → [<u>C</u>] | A. Combining Features |
| 2. Multiplying Rainfall × Soil_Moisture → [<u>A</u>] | B. Embedded Method |
| 3. Random Forest ranks features by importance → [<u>B</u>] | C. Filter Method |

◆ TAKEAWAY

Feature Engineering has two pillars: Creation (build new signals) and Selection (remove noise).
Both run AFTER splitting.

PHASE 7 BASELINE ESTABLISHMENT (THE PERFORMANCE FLOOR)

Before writing code for a complex model, you must answer: What constitutes a “good” prediction? A baseline is the simplest possible model - often a historical average, majority class guess, or primitive linear model.

- **Technical benchmark:** If a more complex machine learning model cannot outperform a simple baseline (such as predicting the historical average or majority class), the additional complexity is not justified.
- **Business benchmark:** Is a 2% increase in prediction accuracy worth spending millions more on cloud infrastructure, training time, and maintenance?

▷ PRACTICE P4 · COST-BENEFIT ANALYSIS

A Decision Tree classifier achieves 80% accuracy. A Random Forest classifier achieves 81% accuracy, but requires significantly more computation and costs an additional ₹20 lakh to deploy.

(a) Is the 1% improvement necessarily worth it? [Yes / No]: No

(b) Why is comparing against a baseline important before selecting a more complex model?

Expected answer: To ensure the additional complexity, cost, and maintenance of the new model are justified by a significant performance gain over a simpler, cheaper alternative.

◆ TAKEAWAY

A baseline sets the performance floor. If the complex model cannot beat it, the extra cost and complexity are wasted.

PHASE 8 THE MODELING LOOP (SELECT, TUNE & TRAIN)

In production, selecting an algorithm, setting its dials, and updating its math do not happen linearly - they operate together in an iterative, experimental loop.

Pillar A: Model Selection (Picking the Right Tool)

▷ **HOOK**

A hospital needs to predict heart attacks. An 85% accurate model explains its reasoning step-by-step. A 92% accurate model is a total black-box. Which do you choose when a wrong prediction could be fatal?

Model selection is never about blindly picking the “most powerful” algorithm. It is an exercise in engineering trade-offs.

Decision Factor	What it Means	Trade-off
Interpretability	Can the model explain its reasoning?	Higher → easier to audit, but often lower accuracy.
Accuracy	How correct are the predictions?	Higher → better predictions, but may be a black-box.
Training Time	How long does it take to train?	Complex models take hours/days on large data.
Scalability	Can it handle millions of rows?	Simple models scale easily; deep learning needs GPUs.

Mapping to Core Computational Tasks:

Task	Goal	Algorithm Examples
Regression	Predict a continuous value	Linear Regression, Random Forest Regressor
Classification	Predict a discrete label	Logistic Regression, SVM, Decision Trees
Clustering	Uncover unlabelled structures	K-Means, DBSCAN

▷ **PRACTICE P5 · HOSPITAL CASE STUDY**

A hospital can choose:

A. 85% accurate but explainable model (clear checklist of reasons)

B. 92% accurate black-box model (no explanation of reasoning)

Which would you choose? Consider safety, liability, and trust:

Expected answer: Model A (85% explainable). In healthcare, lives are at stake; doctors need to understand and trust the reasoning to prevent fatal errors and ensure accountability.

A self-driving car company faces the same choice. Does your answer change? Why?

Expected answer: Usually Model A (explainable) for safety and debugging purposes, though they might demand higher accuracy over time. Trust and auditability are critical in safety-critical systems.

Pillar B: Model Training (The Engine Room)

Model Training is the core phase where an algorithm converts data into automated knowledge. The model reviews training data, calculates its errors, and updates its internal parameters.

Concrete Example - Multiple Linear Regression:

A model predicts crop yield based on rainfall and soil moisture:

$$y = w_0 + w_1x_1 + w_2x_2$$

- y = predicted output (Crop Yield)
- x_1, x_2 = input features (Rainfall, Soil Moisture)
- w_1, w_2 = weights (how much influence each feature has)

- w_0 = intercept (baseline when all inputs are zero)

The weights (w_1, w_2) and intercept (w_0) are the internal parameters. Training finds the exact numerical values that yield the most accurate predictions.

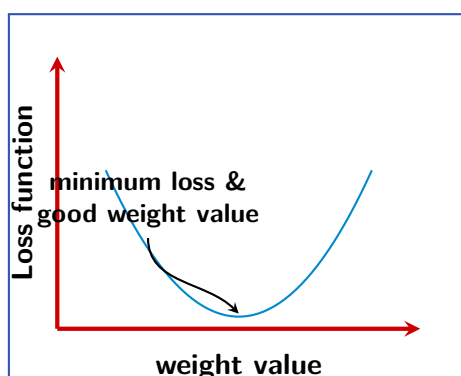
The Measuring Stick: The Loss Function (MSE)

A Loss Function (or Cost Function) calculates a single score measuring how far predictions are from true values:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

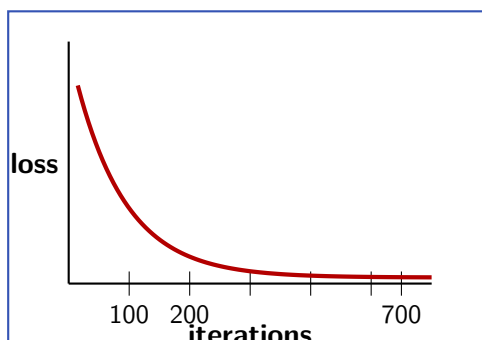
- n = total number of data points
- y_i = actual true value from dataset
- $\hat{y}_i$ = model's predicted value
- We square the error so negatives don't cancel positives. Goal: $\text{MSE} \rightarrow 0$.

Optimization via Gradient Descent:



Imagine standing on a mountain in thick fog. You feel the slope beneath your feet and step downhill. Repeat until the ground flattens. The altitude = Loss Value. Your coordinates = Model Weights. Gradient descent calculates the mathematical slope and adjusts weights toward the lowest error.

Convergence:



As the algorithm approaches the bottom of the error valley, the slope approaches zero. Steps become tiny. The loss flattens. Training is complete and parameters are locked.

The Step-by-Step Training Loop:

Step	Action	What Happens
1. Initialize	Set random parameter values	Weights start as random numbers.
2. Forward Pass	Make predictions	Compute $\hat{y} = w_0 + w_1x_1 + w_2x_2$
3. Compute Loss	Measure error (MSE)	How far are predictions from truth?
4. Backward Pass	Gradient Descent	Calculate slope; adjust weights downhill.
5. Convergence?	Check if loss is stable	If flat $\rightarrow$ stop. If not $\rightarrow$ repeat from Step 2.

▷ PRACTICE P6 · ARRANGE THE TRAINING LOOP

Arrange these steps in the correct execution order (write 1–5):

- [3] Compute Loss (MSE)
- [4] Gradient Descent (Backward Pass)
- [5] Convergence Check
- [2] Forward Pass (Make Predictions)
- [1] Initialize Random Parameters

◆ TAKEAWAY

The Modeling Loop is iterative: Select an algorithm → Set hyperparameters → Train (Forward Pass → Loss → Gradient Descent) → Check convergence → Repeat.

PHASE 9 OFFLINE EVALUATION (THE FINAL EXAM)

Once your model has trained and converged, it is time for the final exam. We pass the untouched Test Set through the locked model to evaluate performance.

1. Technical Evaluation Metrics for Regression:

Metric	What It Does	Best For
MAE (Mean Absolute Error)	Average of $ \text{True} - \text{Predicted} $. Treats all errors linearly.	Interpretable for non-technical stakeholders. MAE of 5.2 = off by 5.2 units on average.
RMSE (Root Mean Squared Error)	Square root of average squared errors. Penalizes large errors heavily.	When large errors are catastrophic. 9 small errors + 1 huge error → RMSE spikes.
R^2 Score (Coefficient of Determination)	Relative scale: 0 to 1. Compares model to baseline average guess.	0.85 = model explains 85% of variance. Directly shows improvement over baseline.

Note: R^2 Score will be covered in detail in the upcoming lectures, so don't worry about this at this point of time.

2. Business Metric Mapping:

In the software industry, mathematical performance must translate into business value. Engineers must map technical results directly to operational impacts.

- **Example:** Reducing RMSE by 1.5 units in agriculture translates to a 12% reduction in predicted crop failure, saving an estimated $\approx 2,00,000$ per field hectare in lost revenue.

▷ PRACTICE P7 · PICK THE RIGHT METRIC

- (a) A medical prediction error of 100 units is catastrophic (fatal).
Which metric? [MAE / RMSE / R^2]: RMSE
- (b) A non-technical CEO asks "how much better is this model than random guessing?"
Which metric? [MAE / RMSE / R^2]: R^2
- (c) A logistics team wants to know "on average, how many km off are our delivery estimates?"
Which metric? [MAE / RMSE / R^2]: MAE

△ REFLECT 2

Why is improving RMSE by 1.5 units valuable to an agricultural business?

Hint: Does it map to reduced crop failure? Increased revenue?

Expected answer: Reducing RMSE by 1.5 units directly translates to fewer extreme prediction errors, which in turn reduces catastrophic crop failures and saves revenue, making the model highly valuable.

◆ TAKEAWAY

MAE for equal penalty interpretation. RMSE when large errors are catastrophic. R^2 for relative performance vs. baseline. Always translate metrics to business value.

PHASE 10 MODEL DEPLOYMENT (INTO THE WILD)

▷ HOOK

You spent months building an accurate model. It runs perfectly on your laptop. But a customer cannot access your terminal. How do you make this mathematical asset usable for thousands of everyday customers?

A model on a laptop is useless to a business. Deployment packages trained parameters into accessible formats and embeds them into production infrastructure.

1. Model Exporting & Serialization:

- .pkl (Pickle): Standard Python format for scikit-learn models.

2. Production Infrastructure Pathways:

Pathway	How It Works
Web Services (APIs)	Wrap model in Flask/FastAPI. Exposes an API endpoint.
Cloud Platforms	Deploy to AWS SageMaker, Google AI Platform, or Azure.
Edge Deployment	Compress model onto local hardware (phone/IoT).

3. Production Data Ingestion Strategies:

Strategy	How	Example	
Batch Predictions	Runs periodically (e.g., nightly) on massive datasets.	Weekly product recommendations for e-commerce.	
Real-Time Inference	Runs instantly on demand, one request at a time.	Fraud detection at checkout.	

◆ TAKEAWAY

Serialize your model, choose the right pathway (API, Cloud, Edge), and pick Batch vs. Real-Time based on latency needs.

PHASE 11 POST-DEPLOYMENT (THE LIVING SYSTEM)

▷ HOOK

You deploy an ML model in 2019 to predict airline travel. In 2020, a global pandemic strikes. Your model was trained on 2010–2019 data. Is it still accurate?

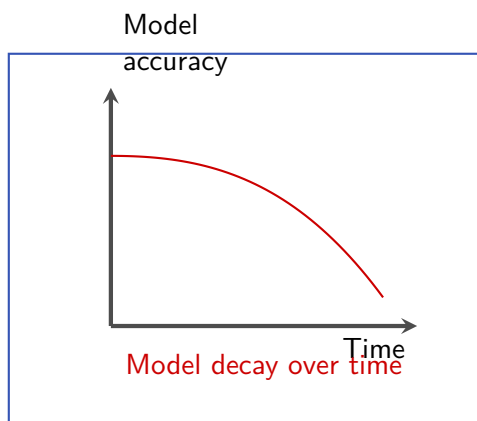
Traditional code runs identically forever. ML models decay. **Model Drift** occurs when prediction accuracy drops because the real-world environment has changed, rendering original training insights outdated.

Continuous Production Monitoring:

- Tools like Prometheus and MLflow run continuously in the background.
- They track statistical distributions of incoming live data.
- They alert engineering teams the moment performance drops below an acceptable baseline threshold.

The Retraining Cycle:

Strategy	How It Works	Example
Periodic Retraining	Fixed schedule (monthly/quarterly), regardless of performance.	Retrain on the 1st of every month automatically.
Triggered Retraining	Starts when monitoring detects accuracy below a threshold.	Alert: accuracy dropped below 85% → retrain now.



▷ PRACTICE P9 · RETRAINING SCENARIOS

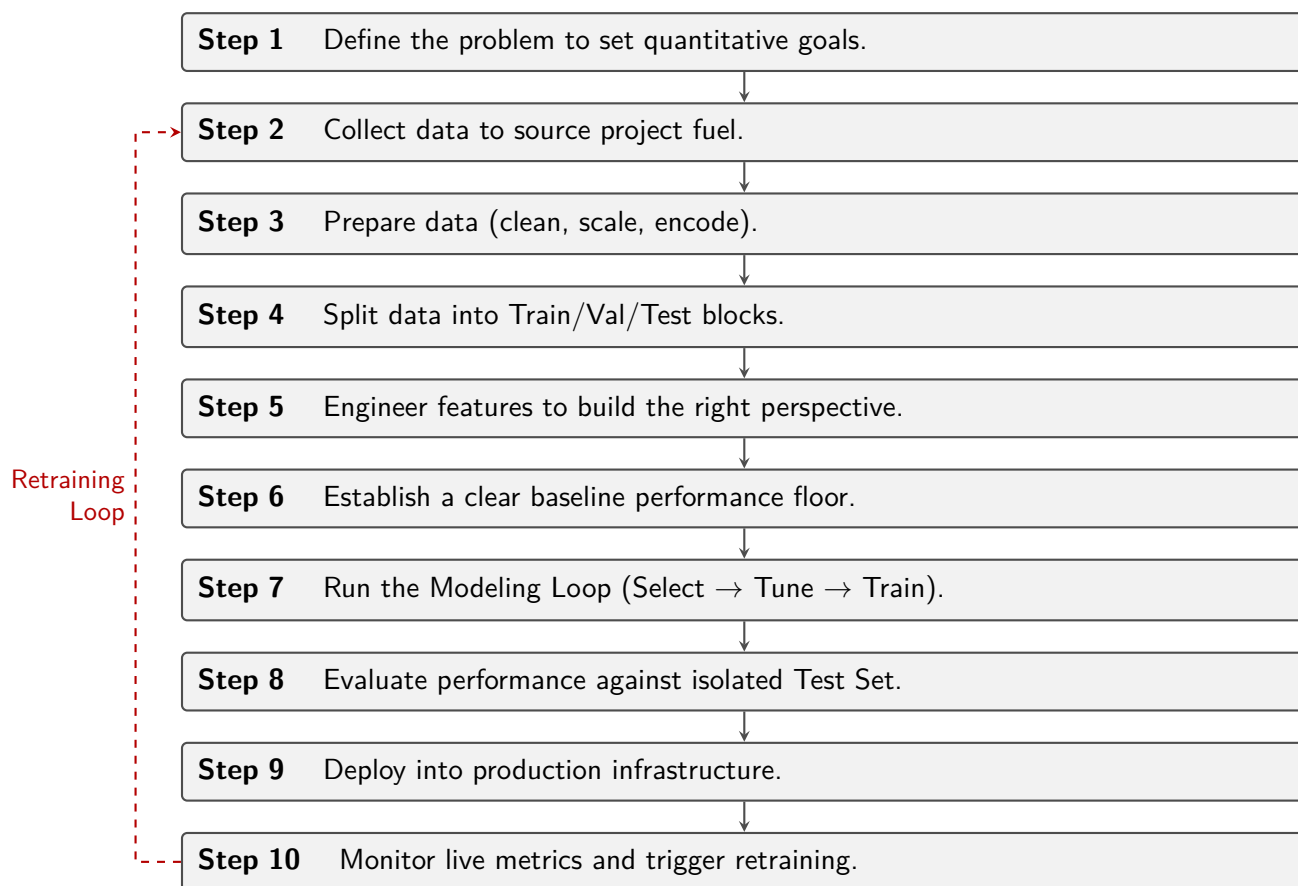
- (a) A company retrains every month regardless of performance. This is:
 A. Triggered Retraining B. Periodic Retraining Answer: [B]
- (b) A monitoring tool detects accuracy dropped from 92% to 78% and auto-retrains. This is:
 A. Triggered Retraining B. Periodic Retraining Answer: [A]
- (c) [True / False] You can deploy a model and walk away without monitoring. [False]

◆ TAKEAWAY

A model is not a static script. It is a living mathematical system that must adapt to a changing world through monitoring and retraining.

THE COMPLETE MACRO LOOP

Every ML system follows this continuous, iterative framework:



▷ PRACTICE P10 · ARRANGE THE LIFECYCLE

Arrange these phases in the correct order (write 1–5):

- [2] Feature Engineering
- [5] Deployment
- [1] Data Splitting
- [3] Baseline Establishment
- [4] Evaluation

▷ PRACTICE P11 · PUTTING IT ALL TOGETHER: CROP YIELD PREDICTION

You are building a crop yield prediction system for an agricultural company. Answer the following questions based on the ENTIRE lifecycle:

1. Which dataset (Train/Val/Test) should remain locked until the very end?

→ Test

2. Create one interaction feature using "Rainfall" and "Sunlight_Hours":

→ New Feature Name: Water_Sun_Index = Rainfall × Sunlight_Hours

3. Your baseline (simple average) predicts yield with MAE of 12. Your model achieves MAE of 11. Training costs ₹15 lakh. Is it worth deploying? Justify:

Expected answer: No. A small MAE improvement (12 to 11) is unlikely to justify the high training and deployment cost of ₹15 lakh.

4. A prediction error of 50 tonnes is catastrophic. Choose the evaluation metric:

[MAE / RMSE / R²]: RMSE

5. You need to predict yields once per growing season for all 10,000 farms. Choose:

[Batch Inference / Real-Time Inference]: Batch Inference

6. After deployment, a severe drought changes weather patterns. What is this called?

→ Model Drift

What retraining strategy would automatically handle this?

→ Triggered Retraining

BY THE END OF THIS SHEET YOU CAN

- ☐ Partition data into Train/Val/Test blocks and explain why splitting precedes Feature Engineering.
- ☐ Apply Feature Creation (Domain Knowledge, Interaction Terms) and Feature Selection (Filter, Embedded).
- ☐ Establish a performance baseline and evaluate its business cost-benefit.
- ☐ Select models based on Interpretability vs. Accuracy trade-offs and map tasks to Regression/Classification/Clustering.
- ☐ Classify items as Parameters vs. Hyperparameters and trace the complete Training Loop.
- ☐ Calculate MSE by hand and choose the right evaluation metric (MAE, RMSE, R^2) for a given scenario.
- ☐ Deploy a serialized model via APIs, Cloud, or Edge and monitor it for Model Drift.